MAT 3007 – Optimization

Integer Optimization

Lecture 26

Instructor: Junchi Yang

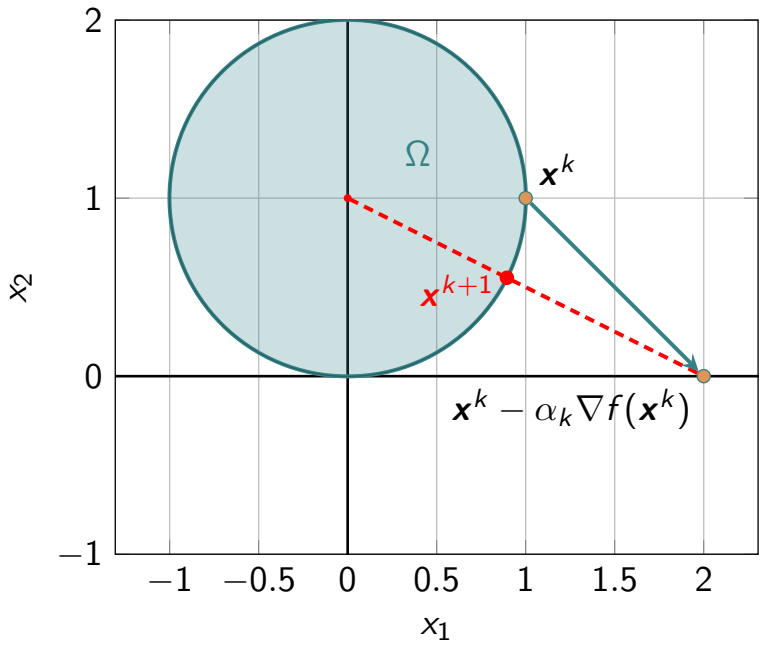
CUHK-Shenzhen

Recap

We consider the following constrained optimization problem:

$$\begin{aligned} \text{minimize}_{\boldsymbol{x}} \quad & f(\boldsymbol{x}) \\ \text{subject to} \quad & \boldsymbol{x} \in \Omega. \end{aligned}$$

Idea of Projected Gradient Descent



Euclidean Projections

Definition: Euclidean Projections

Let $\Omega \subset \mathbb{R}^n$ be a nonempty, closed, convex set.

The (Euclidean) projection of $\boldsymbol{x}$ onto Ω is defined as the unique minimizer $\boldsymbol{y}^*$ of the constrained optimization problem:

$$\min_{\boldsymbol{y}} \ \frac{1}{2}\|\boldsymbol{x}-\boldsymbol{y}\|^2 \quad \text{s.t.} \quad \boldsymbol{y} \in \Omega$$

and we write $\boldsymbol{y}^* = \mathcal{P}_\Omega(\boldsymbol{x})$.

Observation:

Euclidean Projections

Definition: Euclidean Projections

Let $\Omega \subset \mathbb{R}^n$ be a nonempty, closed, convex set.

The (Euclidean) projection of $\boldsymbol{x}$ onto Ω is defined as the unique minimizer $\boldsymbol{y}^*$ of the constrained optimization problem:

$$\min_{\boldsymbol{y}} \ \frac{1}{2}\|\boldsymbol{x}-\boldsymbol{y}\|^2 \quad \text{s.t.} \quad \boldsymbol{y} \in \Omega$$

and we write $\boldsymbol{y}^* = \mathcal{P}_\Omega(\boldsymbol{x})$.

Observation:

- The projection $\boldsymbol{y}^* = \mathcal{P}_\Omega(\boldsymbol{x})$ is the point in Ω that has the minimum distance to $\boldsymbol{x}$.

Remark: Projections

Observation:

- For many sets, explicit formulae for the projections can be derived.
  - Linear Constraint $\Omega = \{\boldsymbol{x} : \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}\}$

Remark: Projections

Observation:

- For many sets, explicit formulae for the projections can be derived.
  - Linear Constraint $\Omega = \{\boldsymbol{x} : \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}\}$
  - Box Constraint $\Omega = \{\boldsymbol{x} \in \mathbb{R}^n : x_i \in [a_i, b_i],\ \forall\ i\} = [\boldsymbol{a}, \boldsymbol{b}]$

Remark: Projections

Observation:

- For many sets, explicit formulae for the projections can be derived.
  - Linear Constraint $\Omega = \{\boldsymbol{x} : \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}\}$
  - Box Constraint $\Omega = \{\boldsymbol{x} \in \mathbb{R}^n : x_i \in [a_i, b_i],\ \forall\ i\} = [\boldsymbol{a}, \boldsymbol{b}]$
  - Ball Constraint $\Omega = \{\boldsymbol{x} \in \mathbb{R}^n : \|\boldsymbol{x} - \boldsymbol{m}\| \leq r\}$

Remark: Projections

Observation:

- For many sets, explicit formulae for the projections can be derived.
  - Linear Constraint $\Omega = \{\boldsymbol{x} : \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}\}$
  - Box Constraint $\Omega = \{\boldsymbol{x} \in \mathbb{R}^n : x_i \in [a_i, b_i],\ \forall\ i\} = [\boldsymbol{a}, \boldsymbol{b}]$
  - Ball Constraint $\Omega = \{\boldsymbol{x} \in \mathbb{R}^n : \|\boldsymbol{x} - \boldsymbol{m}\| \leq r\}$
- In general: an optimization problem needs to be solved to obtain $\mathcal{P}_\Omega(\boldsymbol{x})$.

Optimization Problems with Convex Constraints

Constrained minimization problems of the form:

$$\min_{\boldsymbol{y}} \; f(\boldsymbol{y}) \quad \text{s.t.} \quad \boldsymbol{y} \in \Omega, \tag{1}$$

where $\Omega \subset \mathbb{R}^n$ is a convex and closed set.

Optimization Problems with Convex Constraints

Constrained minimization problems of the form:

$$\min_{\boldsymbol{y}} \; f(\boldsymbol{y}) \quad \text{s.t.} \quad \boldsymbol{y} \in \Omega, \tag{1}$$

where $\Omega \subset \mathbb{R}^n$ is a convex and closed set.

Theorem: FONC for Problems with Convex Constraints

Let f be continuously differentiable on an open set that contains the convex, closed set $\Omega \subset \mathbb{R}^n$. Let $\boldsymbol{y}^* \in \Omega$ be a local minimizer of (1), then:

$$\nabla f(\boldsymbol{y}^*)^\top (\boldsymbol{y} - \boldsymbol{y}^*) \geq 0, \quad \forall\, \boldsymbol{y} \in \Omega. \tag{2}$$

Optimization Problems with Convex Constraints

Constrained minimization problems of the form:

$$\min_{\boldsymbol{y}} \; f(\boldsymbol{y}) \quad \text{s.t.} \quad \boldsymbol{y} \in \Omega, \tag{1}$$

where $\Omega \subset \mathbb{R}^n$ is a convex and closed set.

Theorem: FONC for Problems with Convex Constraints

Let f be continuously differentiable on an open set that contains the convex, closed set $\Omega \subset \mathbb{R}^n$. Let $\boldsymbol{y}^* \in \Omega$ be a local minimizer of (1), then:

$$\nabla f(\boldsymbol{y}^*)^\top (\boldsymbol{y} - \boldsymbol{y}^*) \geq 0, \quad \forall\, \boldsymbol{y} \in \Omega. \tag{2}$$

- If f is convex, then $\boldsymbol{y}^* \in \Omega$ is global minimizer of (1) iff the FONC is satisfied.
- A point satisfying (2) is called a stationary point of (1).

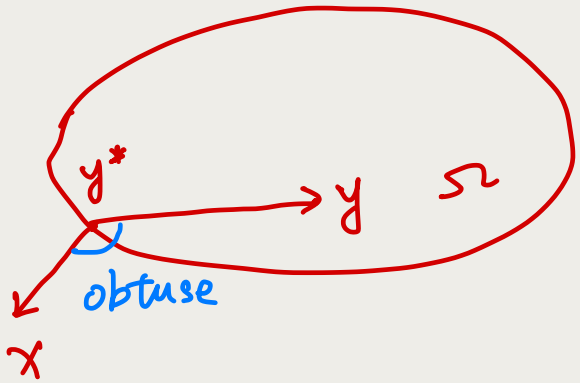
Properties of the Projection Mapping

Projection Theorem

Let Ω be a nonempty, closed, and convex set. Then:

- A point $\boldsymbol{y}^*$ is the projection of $\boldsymbol{x}$ onto Ω, i.e., $\boldsymbol{y}^* = \mathcal{P}_\Omega(\boldsymbol{x})$, if and only if

$$(\boldsymbol{y}^* - \boldsymbol{x})^\top (\boldsymbol{y} - \boldsymbol{y}^*) \geq 0, \quad \forall\, \boldsymbol{y} \in \Omega.$$



proof: y^* is the solution to

$$\min_{y} \frac{1}{2}\|y - x\|^2 \quad \text{s.t } y \in \Omega$$

Then apply the previous theorem.

Properties of the Projection Mapping

Projection Theorem

Let Ω be a nonempty, closed, and convex set. Then:

- A point $\boldsymbol{y}^*$ is the projection of $\boldsymbol{x}$ onto Ω, i.e., $\boldsymbol{y}^* = \mathcal{P}_\Omega(\boldsymbol{x})$, if and only if

$$(\boldsymbol{y}^* - \boldsymbol{x})^\top (\boldsymbol{y} - \boldsymbol{y}^*) \geq 0, \quad \forall\ \boldsymbol{y} \in \Omega.$$

- The mapping $\mathcal{P}_\Omega : \mathbb{R}^n \to \mathbb{R}^n$ is Lipschitz continuous with constant $L = 1$.

$$\| P_\Omega(x) - P_\Omega(y) \| \leq \| x - y \|$$

Properties of the Projection Mapping

Projection Theorem

Let Ω be a nonempty, closed, and convex set. Then:

(1) ▶ A point $\boldsymbol{y}^*$ is the projection of $\boldsymbol{x}$ onto Ω, i.e., $\boldsymbol{y}^* = \mathcal{P}_\Omega(\boldsymbol{x})$, if and only if

$$(\boldsymbol{y}^* - \boldsymbol{x})^\top (\boldsymbol{y} - \boldsymbol{y}^*) \geq 0, \quad \forall\ \boldsymbol{y} \in \Omega.$$

(2) ▶ The mapping $\mathcal{P}_\Omega : \mathbb{R}^n \to \mathbb{R}^n$ is Lipschitz continuous with constant $L = 1$.

(3) ▶ The vector $\boldsymbol{x}^*$ is a stationary point of (1) if and only if

$$\boldsymbol{x}^* - \mathcal{P}_\Omega(\boldsymbol{x}^* - \lambda \nabla f(\boldsymbol{x}^*)) = \boldsymbol{0} \quad \text{for any } \lambda > 0.$$

Proof of (3):

$$x^* = P_\Omega\left(x^* - \lambda \nabla f(x^*)\right)$$

$\Leftrightarrow$ x^* is the solution to $\min_y \frac{1}{2}\left\| y - \left(x^* - \lambda \nabla f(x^*)\right)\right\|^2$ s.t $y \in \Omega$

By (1)
$\Leftrightarrow$ $\left(x^* - \left(x^* - \lambda \nabla f(x^*)\right)\right)^T (y - x^*) \geq 0 \quad \forall y \in \Omega$

$\Leftrightarrow$ $\lambda \nabla f(x^*)^T (y - x^*) \geq 0 \quad \forall y \in \Omega$

$\lambda > 0$
$\Leftrightarrow$ $\nabla f(x^*)^T (y - x^*) \geq 0 \quad \forall y \in \Omega$

□

Motivation & Strategy:

- Setting $\boldsymbol{x}^{k+1} = \boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k)$ might likely generate infeasible iterates: $\boldsymbol{x}^{k+1} \notin \Omega$.
- Idea: We project the step $\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k)$ back onto Ω:

$$\boldsymbol{x}^{k+1} = \mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k)).$$

Remark:

- The algorithm will converge with proper assumptions and step sizes.

We can consider gradient descent as a special case of projected gradient descent with $\Omega = \mathbb{R}^n$

Line Search

If λ_k is determined by line search, every adjustment of λ_k requires to reevaluate $\mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k))$. This can be expensive.

Line Search

If λ_k is determined by line search, every adjustment of λ_k requires to reevaluate $\mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k))$. This can be expensive.

Observation:

$$\boldsymbol{x}^{k+1} = \mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k)) = \boldsymbol{x}^k + [\mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k)) - \boldsymbol{x}^k].$$

Line Search

If λ_k is determined by line search, every adjustment of λ_k requires to reevaluate $\mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k))$. This can be expensive.

Observation:

$$\boldsymbol{x}^{k+1} = \mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k)) = \boldsymbol{x}^k + [\mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k)) - \boldsymbol{x}^k].$$

Consider $\boldsymbol{d}^k = \mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k)) - \boldsymbol{x}^k$ as a search direction. Introduce another step size α_k:

$$\boldsymbol{x}^{k+1} = \boldsymbol{x}^k + \alpha_k \boldsymbol{d}^k = (1 - \alpha_k)\underbrace{\boldsymbol{x}^k}_{\in \Omega} + \alpha_k \underbrace{\mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k))}_{\in \Omega}.$$

Line Search

If λ_k is determined by line search, every adjustment of λ_k requires to reevaluate $\mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k))$. This can be expensive.

Observation:

$$\boldsymbol{x}^{k+1} = \mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k)) = \boldsymbol{x}^k + [\mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k)) - \boldsymbol{x}^k].$$

Consider $\boldsymbol{d}^k = \mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k)) - \boldsymbol{x}^k$ as a search direction. Introduce another step size α_k:

$$\boldsymbol{x}^{k+1} = \boldsymbol{x}^k + \alpha_k \boldsymbol{d}^k = (1 - \alpha_k)\boldsymbol{x}^k + \alpha_k \mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k)).$$

- If $\alpha_k \in [0, 1]$, then the convexity of Ω implies that $\boldsymbol{x}^{k+1}$ will be feasible if $\boldsymbol{x}^k \in \Omega$.
- $\boldsymbol{d}^k$ is a descent direction if $\boldsymbol{x}^k$ is not a stationary point.
- We can do line search on α_k. evaluate $P_\Omega(\cdot)$ only once in each iteration.

Projected Gradient Method with Backtracking Line Search

1. Initialization: Choose an initial point $\boldsymbol{x}^0 \in \Omega$ and $\sigma, \gamma \in (0,1)$.

For $k = 0, 1, \ldots$: if not, do one projection.

2. Select $\lambda_k > 0$ and compute $\nabla f(\boldsymbol{x}^k)$ and the new direction $\boldsymbol{d}^k = \mathcal{P}_\Omega(\boldsymbol{x}^k - \lambda_k \nabla f(\boldsymbol{x}^k)) - \boldsymbol{x}^k$.
3. If $\|\boldsymbol{d}^k\| \leq \lambda_k \varepsilon$, then STOP and $\boldsymbol{x}^k$ is the output.
4. Choose a maximal step size $\alpha_k \in \{1, \sigma, \sigma^2, \ldots\} \subset (0,1]$ that satisfies the Armijo condition

$$f(\boldsymbol{x}^k + \alpha_k \boldsymbol{d}^k) - f(\boldsymbol{x}^k) \leq \gamma \alpha_k \cdot \nabla f(\boldsymbol{x}^k)^\top \boldsymbol{d}^k.$$

5. Set $\boldsymbol{x}^{k+1} = \boldsymbol{x}^k + \alpha_k \boldsymbol{d}^k$.

Theorem: Convergence Projected Gradient Method



Assumptions: f is cont. diff., Ω is nonempty, convex, and closed and the step sizes $\{\lambda_k\}_k$ are bounded:

$$0 < \underline{\lambda} \leq \lambda_k \leq \overline{\lambda} \quad \forall\ k.$$

$\rightsquigarrow$ Every accumulation point of $\{\boldsymbol{x}^k\}$ is a stationary point.

Theorem: Convergence Projected Gradient Method



Assumptions: f is cont. diff., Ω is nonempty, convex, and closed and the step sizes $\{\lambda_k\}_k$ are bounded:

$$0 < \underline{\lambda} \leq \lambda_k \leq \overline{\lambda} \quad \forall\, k.$$

$\rightsquigarrow$ Every accumulation point of $\{\boldsymbol{x}^k\}$ is a stationary point.

If ∇f is additionally Lipschitz continuous, we obtain:

- If f is convex, we converge to global solutions of the problem.
- If $\lambda_k \leq \frac{2(1-\gamma)}{L}$, then $\alpha_k = 1$ will be accepted as step size.
- If f is strongly convex, then $\{\boldsymbol{x}^k\}_k$ converges linearly to a global solution $\boldsymbol{x}^* \in \Omega$.

Example — I

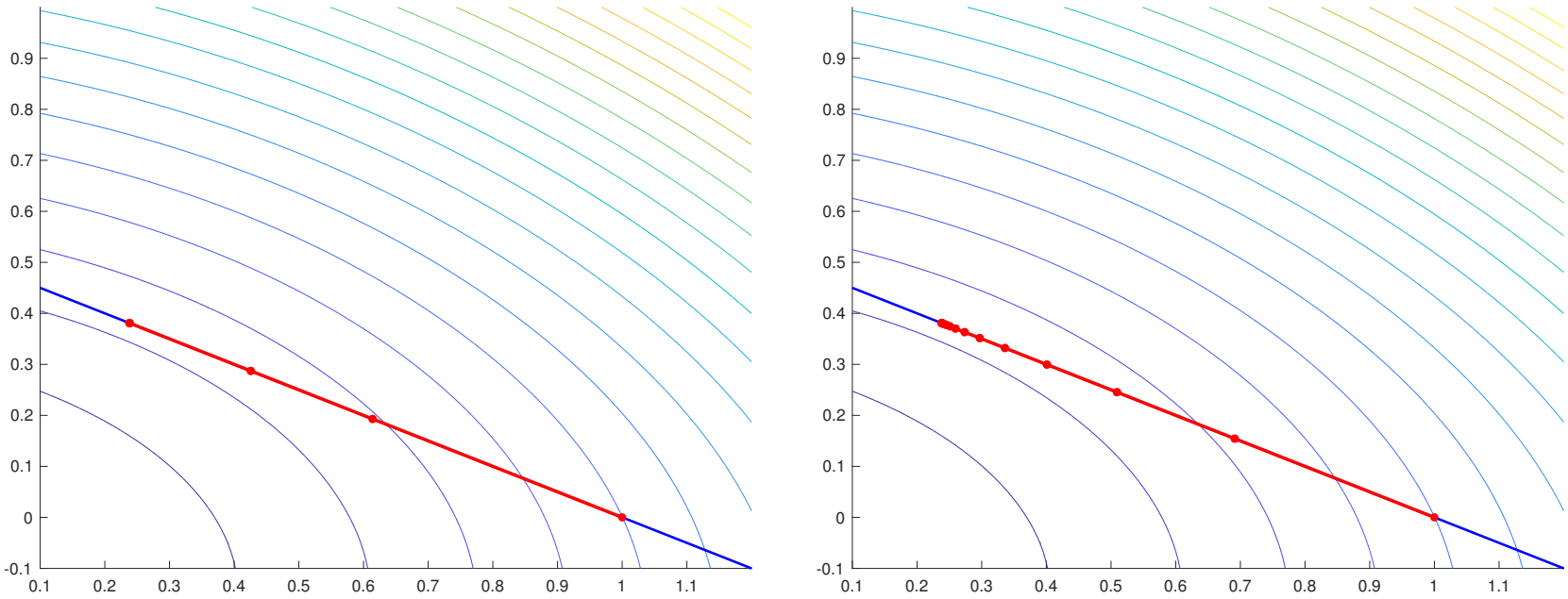
$$\begin{aligned} \text{minimize} \quad & e^{x_1+x_2} + x_1^2 + 3x_2^2 - x_1x_2 \\ \text{subject to} \quad & x_1 + 2x_2 = 1 \end{aligned}$$

Setting $\boldsymbol{A} = [1, 2]$, $b = 1$, and $\Omega = \{\boldsymbol{x} : \boldsymbol{A}\boldsymbol{x} = b\}$, we have

$$\mathcal{P}_\Omega(\boldsymbol{x}) = \begin{bmatrix} 4/5 & -2/5 \\ -2/5 & 1/5 \end{bmatrix} x + \begin{bmatrix} 1/5 \\ 2/5 \end{bmatrix}$$

We use the initial feasible point $\boldsymbol{x}^0 = (1, 0)^\top$ and $\lambda_k = 1$ and $\lambda_k = 0.1$.

Example — I: Results



- **Left:** $\lambda_k = 1$. **Right:** $\lambda_k = 0.1$.

Algorithms

when should we use them?

① bisection , golden section (1-D problem)

↓ f' ↓ f

② gradient descent (high dimensions, gradient available)

↓ ∇f

③ Newton's method (moderate dimensions, gradient, Hessian available)

↓ ∇f, $\nabla^2 f$

④ projected gradient descent (constrained problem)

↓ $P_{\Omega}(\cdot)$, ∇f

unconstrained problem.

Agenda

So far we have only studied continuous optimization problems (both LP and NLP).

- Decision variables can take continuous values (any fractional values).

In the remaining lectures, we are going to study discrete optimization problems.

- Decision variables can only take discrete values.
- We mainly consider integer linear programs.

Integer Programming

Integer Linear Program

An integer linear program (IP) is a linear program with the additional constraint that all variables must be integers:

$$\begin{aligned} \text{minimize}_{\boldsymbol{x}} \quad & \boldsymbol{c}^\top \boldsymbol{x} \\ \text{subject to} \quad & \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b} \\ & \boldsymbol{x} \geq \boldsymbol{0} \\ & \boldsymbol{x} \in \mathbb{Z}^n \end{aligned}$$

standard-form LP

integer constraint

Here, we use $\mathbb{Z}$ to denote the set of integers.

Integer Linear Program

An integer linear program (IP) is a linear program with the additional constraint that all variables must be integers:

$$\begin{aligned} \text{minimize}_{\boldsymbol{x}} \quad & \boldsymbol{c}^\top \boldsymbol{x} \\ \text{subject to} \quad & \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b} \\ & \boldsymbol{x} \geq \boldsymbol{0} \\ & \boldsymbol{x} \in \mathbb{Z}^n \end{aligned}$$

Here, we use $\mathbb{Z}$ to denote the set of integers.

- One may also encounter mixed integer programs (MIP), in which one set of variables must be integer and the rest are allowed to be continuous.

A Special Case: Binary IP

One subclass of IP is the binary integer program, in which each decision variable has to be either 0 or 1:

$$\begin{aligned} \text{minimize}_{\boldsymbol{x}} \quad & \boldsymbol{c}^\top \boldsymbol{x} \\ \text{s.t.} \quad & \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b} \\ & x_i \in \{0, 1\} \quad \forall\, i. \end{aligned}$$

- The variables in a binary IP are also called indicator variables.

Integer Programs

In many real applications, the decision variables should actually be integers (or binary variables):

- Production planning.
- Shortest path.

In the previous discussions, we ignored those integer constraints for the ease of analysis.

However, it is important to study such problems since they capture many practical situations.

Examples & Modeling

Example: Knapsack Problem

John is planning a trip. There are n items he would like to bring.

- The ith item has value v_i.
- The weight of ith item is a_i.
- His bag has a maximum allowable weight C.
- He wants to bring as much value as possible.

Example: Knapsack Problem

John is planning a trip. There are n items he would like to bring.

- The ith item has value v_i.
- The weight of ith item is a_i.
- His bag has a maximum allowable weight C.
- He wants to bring as much value as possible.

Decision Variables:

- x_i: whether to bring ith item or not: $x_i \in \{0, 1\}$.

Example: Knapsack Problem

John is planning a trip. There are n items he would like to bring.

- The ith item has value v_i.
- The weight of ith item is a_i.
- His bag has a maximum allowable weight C.
- He wants to bring as much value as possible.

Decision Variables:

- x_i: whether to bring ith item or not: $x_i \in \{0, 1\}$.

Optimization Problem:

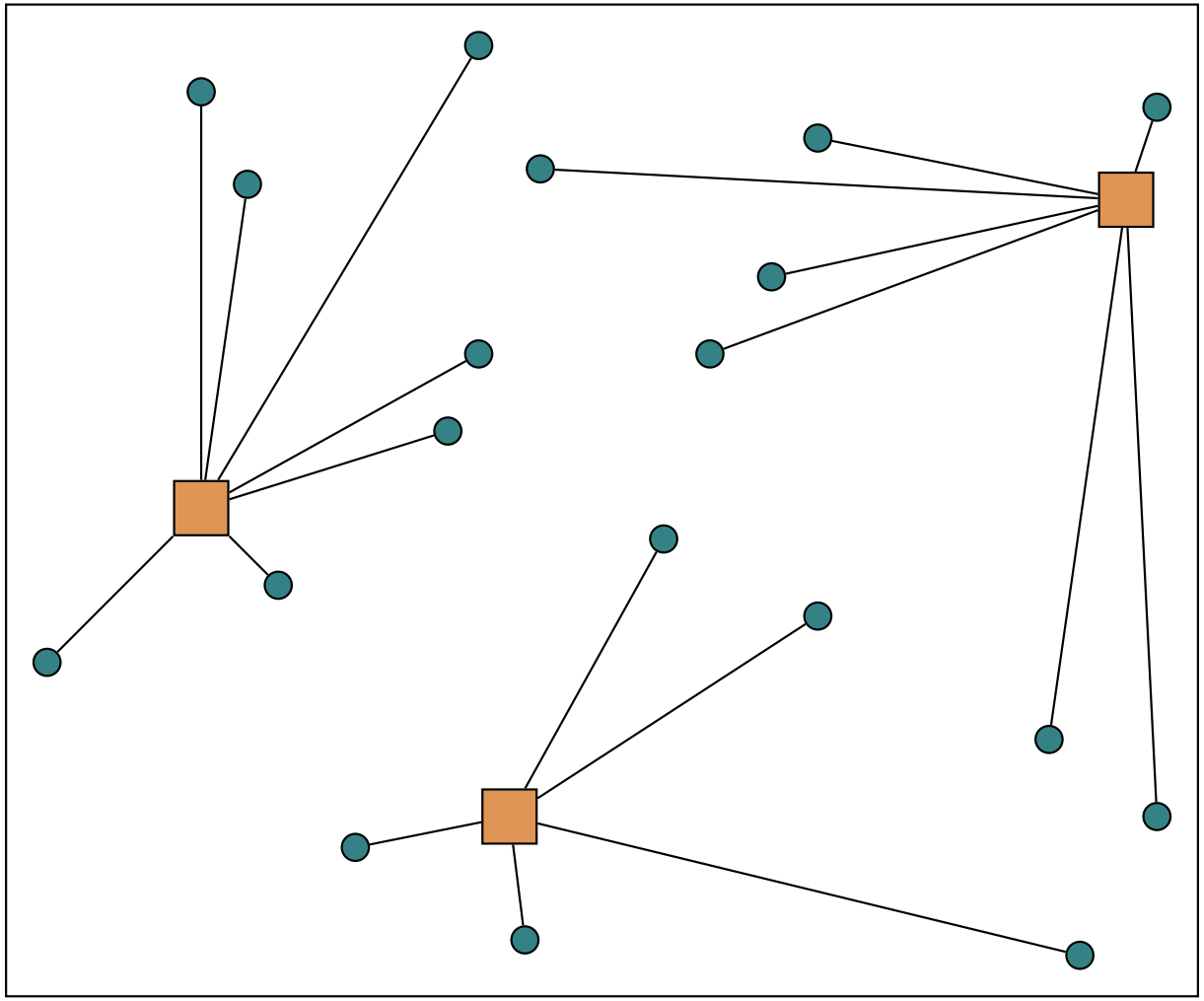
$$\begin{aligned} \text{maximize}_{\boldsymbol{x}} \quad & \sum_{i=1}^{n} v_i x_i \\ \text{subject to} \quad & \sum_{i=1}^{n} a_i x_i \leq C \\ & x_i \in \{0, 1\} \quad \forall\ i. \end{aligned}$$

$\rightsquigarrow$ This is a binary optimization problem.

Example: Warehouse/Facility Location Problem

- There are n warehouses available for use.
- Opening warehouse i has a fixed operating costs f_i.
- There are m customers.
- Customer j has demand d_j. It costs c_{ij} to ship one unit of product from warehouse i to customer j.
- The objective is to satisfy all customers' demands, while minimizing the total costs (operating + shipment).

Facility Location Problem



Decision Variables:

- y_i: whether to open warehouse i or not: $y_i \in \{0, 1\}$.
- x_{ij}: how many units to ship from warehouse i to customer j.

Decision Variables:

- y_i: whether to open warehouse i or not: $y_i \in \{0, 1\}$.
- x_{ij}: how many units to ship from warehouse i to customer j.

Objective function:

$$\text{minimize}_{\boldsymbol{x},\boldsymbol{y}} \quad \sum_{i=1}^{n} f_i y_i + \sum_{i,j} c_{ij} x_{ij}$$

Decision Variables:

- y_i: whether to open warehouse i or not: $y_i \in \{0, 1\}$.
- x_{ij}: how many units to ship from warehouse i to customer j.

Objective function:

$$\text{minimize}_{\boldsymbol{x},\boldsymbol{y}} \quad \sum_{i=1}^{n} f_i y_i + \sum_{i,j} c_{ij} x_{ij}$$

Constraints:

- Need to satisfy every customer's demand: $\sum_{i=1}^{n} x_{ij} \geq d_j$.
- A location can provide supply only if it is opened.
- To capture this, we can define constraints (why?):

$$x_{ij} \leq d_j y_i \quad \text{and} \quad x_{ij} \geq 0.$$

if $y_i = 0$, $x_{ij} = 0$ $\forall j$

if $y_i = 1$, $x_{ij} \leq d_j$

Full Formulation

We can formulate the warehouse/facility location problem as follows:

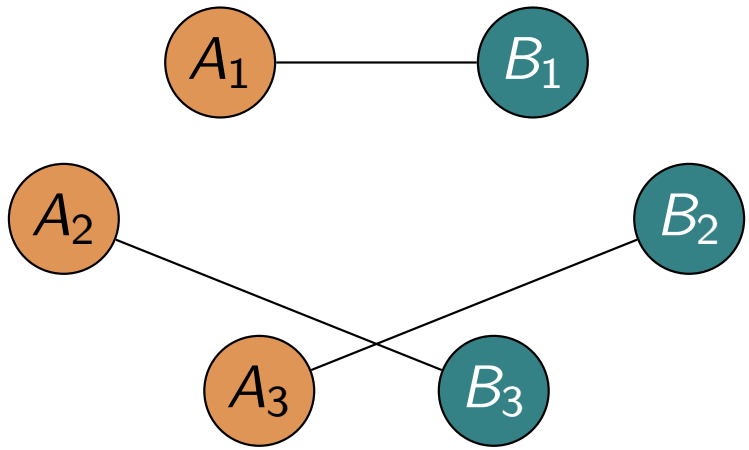
$$
\begin{aligned}
\text{minimize}_{\boldsymbol{x},\boldsymbol{y}} \quad & \textstyle\sum_{i=1}^{n} f_i y_i + \sum_{i,j} c_{ij} x_{ij} & \\
\text{subject to} \quad & \textstyle\sum_{i=1}^{n} x_{ij} \geq d_j, & \text{for all } j \\
& x_{ij} \leq d_j y_i, & \text{for all } i, j \\
& x_{ij} \geq 0 & \\
& y_i \in \{0, 1\} &
\end{aligned}
$$

$\rightsquigarrow$ This is a mixed integer program.

Example: Matching Problem

Setup: Assume there are two groups of individuals registered on a dating platform, labeled Group A and Group B. Each group consists of n individuals.

- Each individual i from Group A rates each individual j from Group B with a score v_{ij}.
- The goal is to find a one-to-one matching between the groups that maximizes the total score.



Matching Problem

Decision Variables:

- x_{ij}: Binary variable indicating whether individual i from Group A is matched with individual j from Group B.

Matching Problem

Decision Variables:

- x_{ij}: Binary variable indicating whether individual i from Group A is matched with individual j from Group B.

Objective Function:

$$\text{maximize} \quad \sum_{i=1}^{n} \sum_{j=1}^{n} v_{ij} x_{ij}$$

Matching Problem

Decision Variables:

- x_{ij}: Binary variable indicating whether individual i from Group A is matched with individual j from Group B.

Objective Function:

$$\text{maximize} \quad \sum_{i=1}^{n} \sum_{j=1}^{n} v_{ij} x_{ij}$$

Constraints:

- Each individual in Group A is matched to exactly one person in Group B: $\sum_{j=1}^{n} x_{ij} = 1, \quad \forall i \in \{1, \ldots, n\}$.
- Each individual in Group B is matched to exactly one person in Group A: $\sum_{i=1}^{n} x_{ij} = 1, \quad \forall j \in \{1, \ldots, n\}$.
- Binary assignment constraint: $x_{ij} \in \{0, 1\}, \quad \forall i, j$.

Matching Problem: Full Formulation

$$\begin{aligned} \text{maximize} \quad & \sum_{i=1}^{n}\sum_{j=1}^{n} v_{ij}x_{ij} \\ \text{s.t.} \quad & \sum_{i=1}^{n} x_{ij} = 1 \quad \forall\, j, \quad \sum_{j=1}^{n} x_{ij} = 1 \quad \forall\, i \\ & x_{ij} \in \{0,1\} \end{aligned}$$

It is a binary optimization problem.

Matching Problem: Full Formulation

$$\begin{aligned} \text{maximize} \quad & \sum_{i=1}^{n}\sum_{j=1}^{n} v_{ij}x_{ij} \\ \text{s.t.} \quad & \sum_{i=1}^{n} x_{ij} = 1 \quad \forall\, j, \quad \sum_{j=1}^{n} x_{ij} = 1 \quad \forall\, i \\ & x_{ij} \in \{0,1\} \end{aligned}$$

It is a binary optimization problem.

- It is also used in e.g., search engine adwords allocation problem, object tracking, and many other useful problems.

Logical Constraints

Integer variables can be used to model logical constraints.

- Binary variables $\{0, 1\}$ correspond to true or false decisions.
- For $x, y \in \{0, 1\}$, the constraint "$x + y = 1$" means: "only one of the variables x or y is 1/true".

Logical Constraints

Integer variables can be used to model logical constraints.

- Binary variables $\{0, 1\}$ correspond to true or false decisions.
- For $x, y \in \{0, 1\}$, the constraint "$x + y = 1$" means: "only one of the variables x or y is 1/true".
- Suppose we have m many constraints: $g_i(x) \leq 0$, $i = 1, \ldots, m$ and at least k many of those constraints should be satisfied.

Logical Constraints

Integer variables can be used to model logical constraints.

- Binary variables $\{0, 1\}$ correspond to true or false decisions.
- For $x, y \in \{0, 1\}$, the constraint "$x + y = 1$" means: "only one of the variables x or y is 1/true".
- Suppose we have m many constraints: $g_i(x) \leq 0$, $i = 1, \ldots, m$ and at least k many of those constraints should be satisfied.

$\rightsquigarrow$ We introduce $y_i \in \{0, 1\}$ and modify the constraints to

$$g_1(x) \leq My_1, \quad \ldots \quad g_m(x) \leq My_m, \quad \sum_{i=1}^{m} (1 - y_i) \geq k,$$

where M is a sufficiently large parameter. at least k of $y_i = 0$

- If $g_i(x) > 0$, then $y_i = 1$. Otherwise, if $g_i(x) \leq 0$, we can choose $y_i \in \{0, 1\}$.

Solving Integer Programs

Our Objective

We have seen that integer optimization problems arise naturally from many applications.

We want to answer two questions in our study:

- How different is it from linear optimization problems?
- How to solve IPs? Are there any interesting properties?

Our Objective

We have seen that integer optimization problems arise naturally from many applications.

We want to answer two questions in our study:

- How different is it from linear optimization problems?
- How to solve IPs? Are there any interesting properties?

We can give a quick answer to the first question:

- It is very different.
- Remember we have defined the complexity class NP-hard to denote a class of hard problems. Integer problems are NP-hard. It is unlikely to have polynomial-time algorithms.
- Even to verify if a set (polyhedron) contains an integer point is very hard.

An Intuitive Thought: Relaxation

General Idea:

Maybe we can first relax the integer constraints, solve the linear program (which is easy) and find some integer solution near the optimal solution of the linear program.

An Intuitive Thought: Relaxation

General Idea:

Maybe we can first relax the integer constraints, solve the linear program (which is easy) and find some integer solution near the optimal solution of the linear program.

- We call the corresponding linear program the LP relaxation of the IP.
- Definitely a plausible method.
- May work well in some case.
- But may fail in other cases.

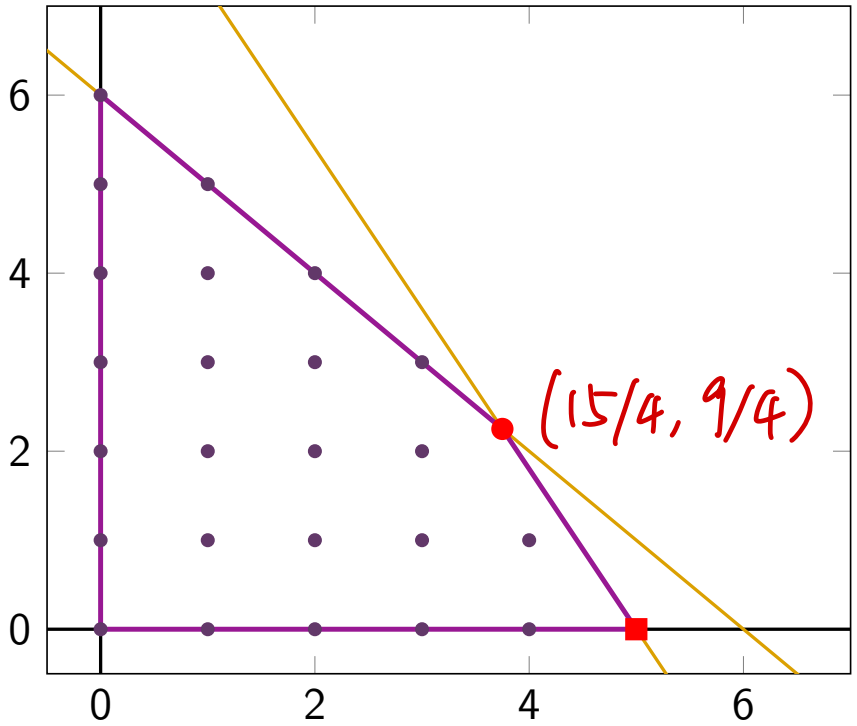
Example: Relaxation

Consider the following integer program (a production planning problem):

$$\begin{aligned} \text{maximize} \quad & 8x_1 + 5x_2 \\ \text{subject to} \quad & 9x_1 + 5x_2 \leq 45 \\ & x_1 + x_2 \leq 6 \\ & x_1, x_2 \geq 0 \\ & x_1, x_2 \in \mathbb{Z} \end{aligned}$$

- If we consider the LP relaxation, the optimal solution is $(x_1, x_2) = (15/4, 9/4)$.

Example: Relaxation



- If we try to round it to the nearest integer solution, we get $(4, 2)$ which is not even feasible for the integer program.
- If we consider the nearest feasible integer point, it will be $(3, 2)$, the objective value is 34.
- The optimal solution to the integer program is actually $(5, 0)$, the objective value is 40.

Another Example: Knapsack Problem

There are three items:

- Item A: 3kg with value 300. x_1
- Item B: 2kg with value 180. x_2
- Item C: 2kg with value 170. x_3

The capacity of the bag is 4kg.

$$\max \quad 300x_1 + 180x_2 + 170x_3$$
$$\text{s.t} \quad 3x_1 + 2x_2 + 2x_3 \leq 4$$
$$x_i \in \{0, 1\} \quad \forall i$$

Another Example: Knapsack Problem

There are three items:

- Item A: 3kg with value 300.
- Item B: 2kg with value 180.
- Item C: 2kg with value 170.

The capacity of the bag is 4kg.

- The relaxation of a binary constraint $x \in \{0, 1\}$ is $x \in [0, 1]$.
- Optimal solution to LP relaxation: (1, 1/2, 0) with optimal value 390.
- Nearest feasible integer solution: (1, 0, 0) with value 300.
- Optimal integer solution: (0, 1, 1) with value 350.

Summary

The optimal solution to the integer program might not be close to the optimal solution to the LP relaxation.

- Directly rounding the LP solution to integers may not yield a good solution ($\rightsquigarrow$ might even be infeasible).
- The reason is the very combinatorial nature of the IP problem (no continuity in the solution space).

But does the LP relaxation provide any useful information?

Today's lecture

- Intro to Integer Optimization

Next lecture

- Solve Integer Optimization Problem

Questions?